

Vercel Deployment Guide

A Step-by-Step Guide for React E-commerce Websites

1. Introduction

Vercel is the premier cloud hosting platform for frontend frameworks, particularly React. It provides ultra-fast global delivery via its Edge Network and supports automated pipelines for continuous integration and delivery (CI/CD). This guide explains how to deploy your React e-commerce website on Vercel using two methods: command-line deployment and automated Git integration.

2. Essential Configurations for React Router

React e-commerce websites use client-side routing (e.g., React Router). By default, if a user directly accesses a route other than the homepage (like `/cart` or `/products`), the hosting server will return a 404 error because the path does not exist as a physical file on the server. To fix this, we must instruct Vercel to redirect all traffic to the main entry point `index.html`.

Create a **vercel.json** file in the root of your frontend project containing:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

3. Bypassing Compilation Warnings (CI=false)

By default, Vercel sets the environment variable **CI=true**. Under this setting, standard static compilers like Create React App will treat ESLint and other compilation warnings as fatal errors, causing the build to fail. Since static sites frequently throw minor warnings (e.g. unused variables), you can configure Vercel to bypass them by prefixing your build command with **CI=false**.

Update your **vercel.json** to include a custom build command:

```
{
  "buildCommand": "CI=false npm run build",
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

4. Method A: CLI Deployment (Terminal)

This method compiles and uploads your local files directly from your developer machine. It is fast and does not require connecting your personal GitHub account to the Vercel cloud dashboard.

1. Open your terminal and navigate to the project folder (e.g. *frontend*).
2. Run the Vercel executable via npx (downloads and runs Vercel CLI without global installation):

```
npx vercel
```

3. Follow the CLI login link generated in the terminal (visit the URL, select login, and authorize the device).
4. Answer the configuration prompts. To accept all defaults, simply pass the **--yes** flag:

```
npx vercel --yes
```

5. Promote the deployment to production to obtain your primary domain name:

```
npx vercel --prod --yes
```

5. Method B: GitHub Integration (Recommended)

This is the recommended approach. Every time you push changes to your GitHub *main* branch, Vercel automatically triggers a build and publishes the updates to your production domain.

1. Push your local modifications to GitHub:
git add .
git commit -m "description"
git push origin main
2. Log in to your account at <https://vercel.com>.
3. Click **Add New > Project**.
4. Search for your repository and click **Import**.
5. Expand the **Build and Development Settings** dropdown.
6. In the **Build Command** field, toggle the override switch and enter: **CI=false npm run build**.
7. Click **Deploy**. Vercel will complete the build and assign a permanent live URL.

TROUBLESHOOTING TIP:

If you encounter a 'Module not found' error during deployment, make sure all modules imported in your code (like **axios** or **react-icons**) are explicitly declared in your project's local *package.json* dependencies list rather than the parent workspace folder.